Conditional Statements

If Statements

- There are the following variants of if statement in C language.
- If statement
- If-else statement
- If else-if ladder
- Nested if

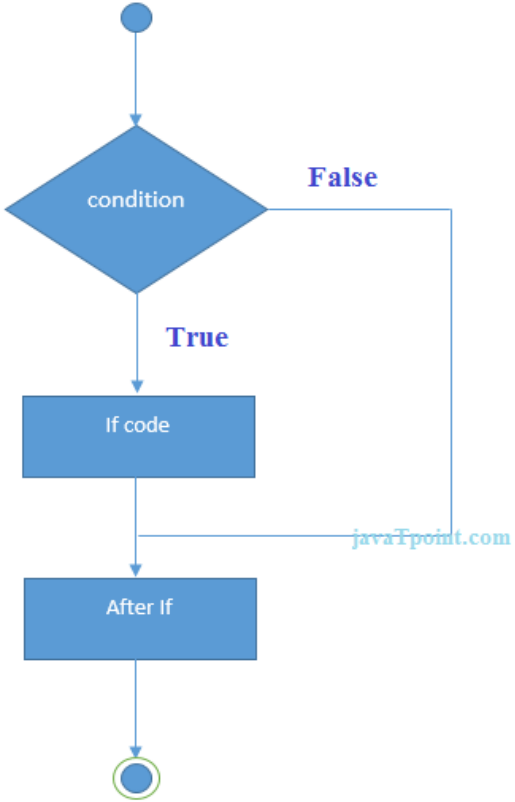
Simple if

- The if statement is used to check some given condition and perform some operations depending upon the correctness of that condition.
- It is mostly used in the scenario where we need to perform the different operations for the different conditions.

Simple if

Syntax

```
if(expression){
//code to be executed
}
```



Simple if

```
#include<stdio.h>
int main()
{
        int number=0;
        printf("Enter a number:");
        scanf("%d",&number);
        if(number%2==0)
        {
        printf("%d is even number",number);
        }
        return 0;
}
```

Simple if

```
#include <stdio.h>
int main()
{
    int a, b, c;
     printf("Enter three numbers?");
    scanf("%d %d %d",&a,&b,&c);
    if(a>b && a>c)
    {
        printf("%d is largest",a);
    }
    if(b>a  && b > c)
    {
        printf("%d is largest",b);
    }
    if(c>a && c>b)
    {
        printf("%d is largest",c);
    }
    if(a == b && a == c)
    {
        printf("All are equal");
    }
    return 0;
}
```

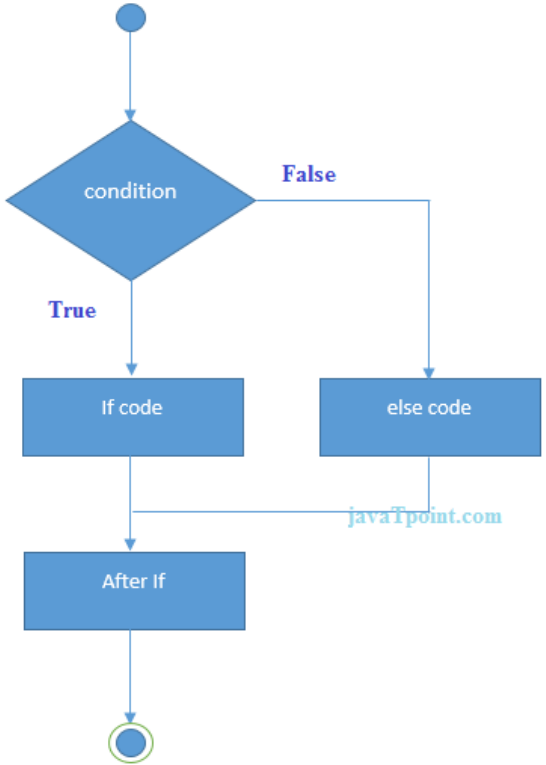
If-else Statement

- The if-else statement is used to perform two operations for a single condition.
- The if-else statement is an extension to the if statement using which, we can perform two different operations, i.e., one is for the correctness of that condition, and the other is for the incorrectness of the condition.
- Here, we must notice that if and else block cannot be executed simulteneously.
- Using if-else statement is always preferable since it always invokes an otherwise case with every if condition

If-else Statement

Syntax

```
if(expression)
{
//code to be executed if condition is true
}
else
{
//code to be executed if condition is false
}
```



If-else Statement

```
#include<stdio.h>
int main()
{
        int number=0;
        printf("enter a number:");
        scanf("%d",&number);
        if(number%2==0)
        {
                printf("%d is even number", number);
        }
        else
        {
                printf("%d is odd number", number);
        }
        return 0;
}
```

Conditional or Ternary Operator (?:) in C

The **conditional operator in C** is kind of similar to the if-else statement as it follows the same algorithm as of if-else statement

But the conditional operator takes less space and helps to write the if-else statements in the shortest way possible.

It is also known as the **ternary operator in C** as it operates on three operands.

Conditional or Ternary Operator (?:) in C

Syntax:

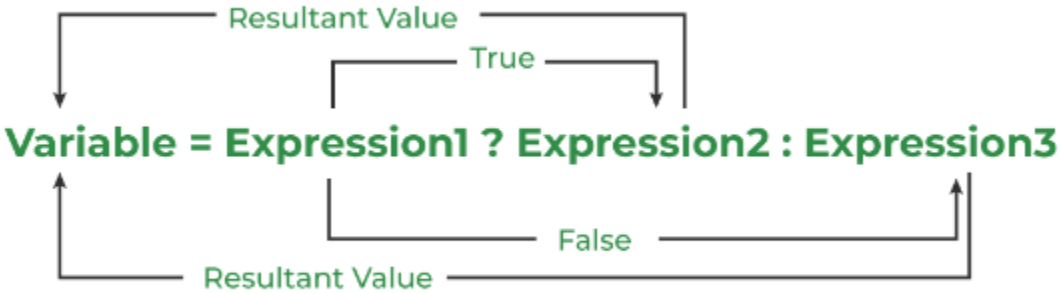
variable = Expression1 ? Expression2 : Expression3;

OR

variable = (condition) ? Expression2 : Expression3;

OR

(condition) ? (variable = Expression2) : (variable = Expression3);



It is visualised as an if-else statement as

```
if(Expression1)
{
   variable = Expression2;
}
else
{
   variable = Expression3;
}
```

Conditional or Ternary Operator (?:) in C

Working:

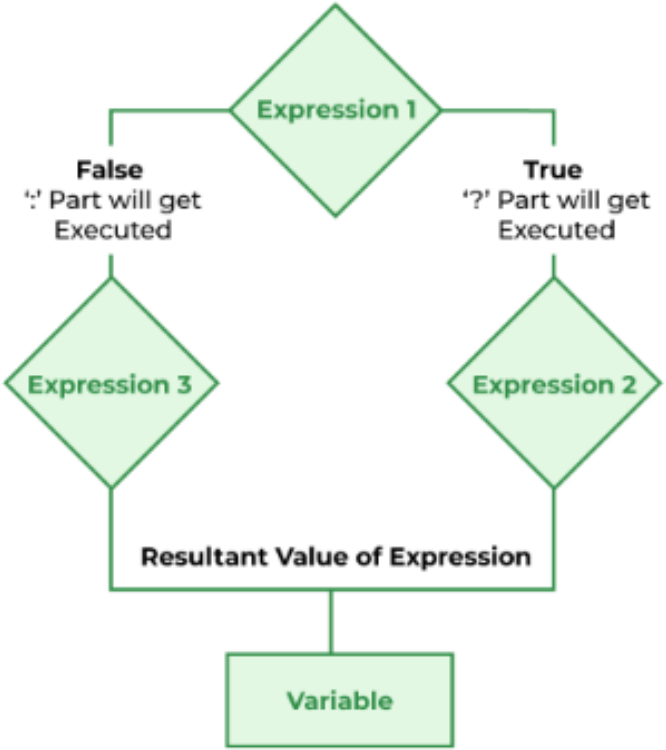
Steps:

1 : Expression1 is the condition to be evaluated.

2A : If the condition(**Expression1**) is True then **Expression2** will be executed.

2B : If the condition(**Expression1**) is false then **Expression3** will be executed.

3 : Results will be returned.



Conditional or Ternary Operator (?:) in C

Program to Store the greatest of the two Numbers using the ternary operator

```
#include <stdio.h>
int main()
{
    int m = 5, n = 4;
    (m > n) ? printf("m is greater than n that is %d > %d", m, n) : printf("n is greater than m that is %d >
                                                                                %d", n, m);
    return 0;
}
```

Program to check whether a number is Even or Odd

```
#include <stdio.h>
int main()
{
   int number = 3;
   (number % 2 == 0) ? printf("Even Number") : printf("Odd Number");
   return 0;
}
```

printf("%d", (num % 2 == 1) ? 0 : 1);

Conditional or Ternary Operator (?:) in C

Associativity of Conditional Operator in C

The associativity of the conditional operator is said to be from right to left. Suppose there are multiple conditional operators in an expression. The expression will be evaluated from right to left.

For example,

- Condition1 ? expression1 : Condition2 ? expression2: expression3

Eg:

ninjaVar1 > ninjaVar2 ? ninjaVar1 : ninjaVar2 > ninjaVar3 ? ninjaVar2: ninjaVar3

1. b=((a==5)?(3):(2));
2. int num = 50;
   char* result = (num < 0) ? "Negative" : (num == 0) ? "Zero" : "Positive";
3. ans=(a==5)?(b==12?4:8):12;
3. Write a C program to find maximum between two numbers using conditional operator.
4. Write a C program to find maximum between three numbers using conditional operator.
5. Write a C program to check whether a number is even or odd using conditional operator.
6. Write a C program to check whether year is leap year or not using conditional operator.
7. Write a C program to check whether character is an alphabet or not using conditional operator.

In the expression above, we have two conditional operators. According to the associativity rule, first, the rightmost conditional operator will be evaluated. Therefore the expression **'ninjaVar2 > ninjaVar3 ? ninjaVar2: ninjaVar3'** will be evaluated first. The leftmost conditional operator '**ninjaVar1 > ninjaVar2 ? ninjaVar1** will be evaluated using the result of **'ninjaVar2 > ninjaVar3 ? ninjaVar2 : ninjaVar3' .**

If-else Vs conditional operator

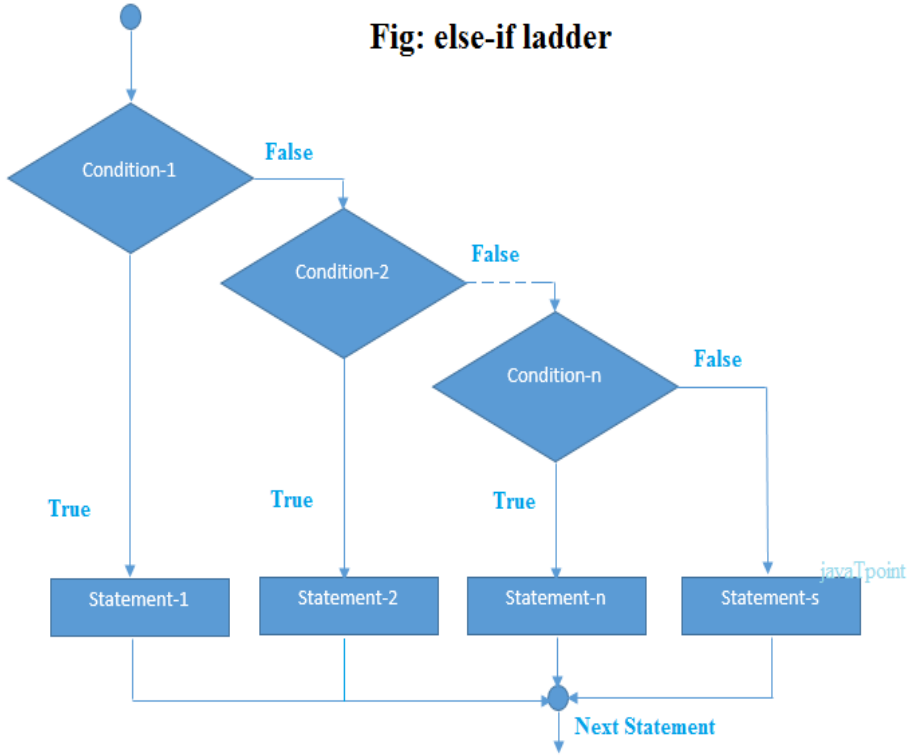
Pointers	Conditional Operator	If-Else Statement
Programming Structure	Single statement	Block of statements
Usage for Assignments	It can be used for assignments	It cannot be used for assignments
Handling Multiple Statements	Not suitable for executing multiple statements	Suitable for executing multiple statements
Nesting Complexity	Nested conditional operators can be complex and hard to debug	Nested if-else statements are easier to read and maintain

If else-if ladder Statement

- The if-else-if ladder statement is an extension to the if-else statement.
- It is used in the scenario where there are multiple cases to be performed for different conditions.
- In if-else-if ladder statement, if a condition is true then the statements defined in the if block will be executed, otherwise if some other condition is true then the statements defined in the else-if block will be executed, at the last if none of the condition is true then the statements defined in the else block will be executed.
- There are multiple else-if blocks possible. It is similar to the switch case statement where the default is executed instead of else block if none of the cases is matched.

If else-if ladder Statement

```
if(condition1)
{
//code to be executed if condition1 is true
}
else if(condition2)
{
//code to be executed if condition2 is true
}
else if(condition3)
{
//code to be executed if condition3 is true
}
...
else
{
//code to be executed if all the conditions are false
}
```



If else-if ladder Statement

```
#include <stdio.h>
int main()
{
    int marks;
    printf("Enter your marks?");
    scanf("%d", &marks);
    if(marks > 85 && marks <= 100)
    {
        printf("Congrats ! you scored grade
A ...");
    }
    else if (marks > 60 && marks <= 85)
    {
        printf("You scored grade B + ...");
    }
    else if (marks > 40 && marks <= 60)
    {
        printf("You scored grade B ...");
    }
    else if (marks > 30 && marks <= 40)
    {
        printf("You scored grade C ...");
    }
    else
    {
        printf("Sorry you are fail ...");
    }
return 0;
}
```

Nested if-else

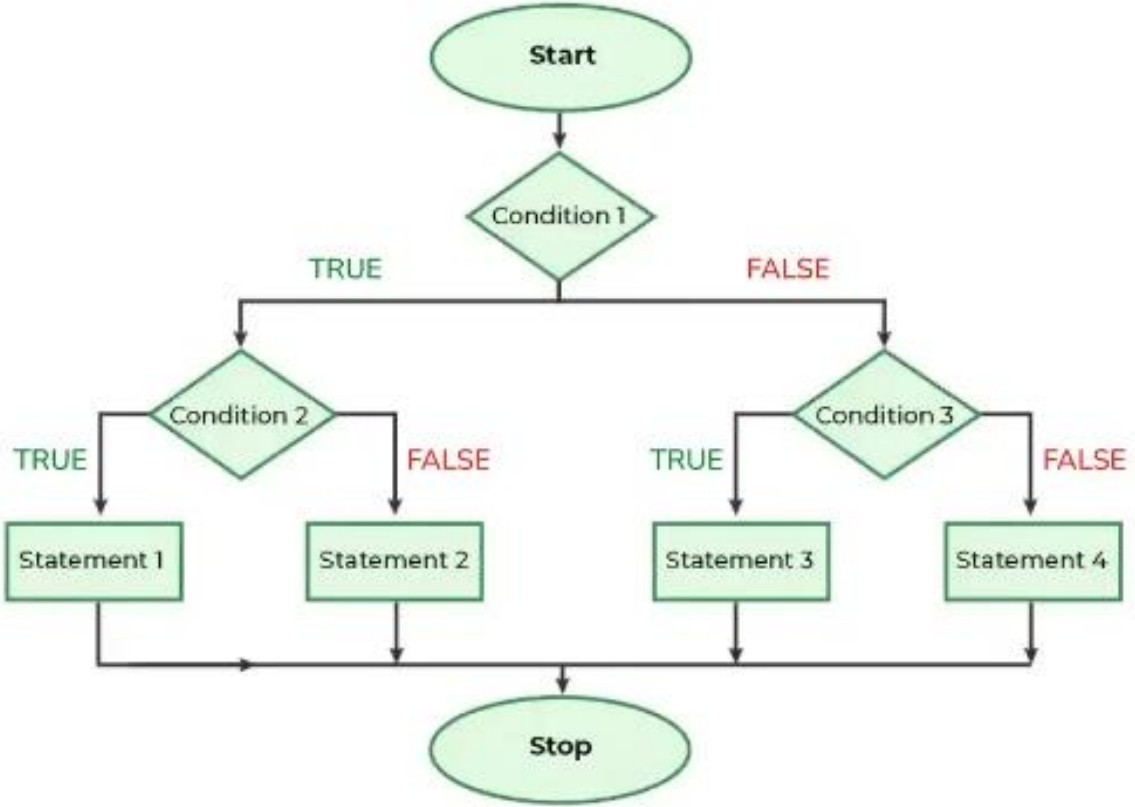
- A nested if in C is an if statement that is the target of another if statement.
- Nested if statements mean an if statement inside another if statement.

Nested if-else

Syntax

```
if (condition1)
{
    // Executes when condition1 is true
    if (condition_2)
    {
        // statement 1
    }
    else
    {
        // Statement 2
    }
}
else
{
    if (condition_3)
    {
        // statement 3
    }
    else
    {
        // Statement 4
    }
}
```

Nested if-else



Nested if-else

Problem Statement: *Determine the grade of a student based on their score. If the score is above 90, it's an 'A'. If the score is between 80 and 90, check if it's above 85 for an 'A-' or else 'B+'. Below 80, if it's above 70, it's a 'C', otherwise, it's a 'D'.*

```
#include  <stdio.h>
int main()
{
    int score = 88;
    // Check if score is greater than 90
    if (score  >  90)
   {
        printf("Grade: A\n");
    }
    else
   {
    // Score is 90 or below, check if score is greater than 80
        if (score  >  80)
        {
  // Score is between 81 and 90, check if score is above 85
            if (score  >  85)
            {
                printf("Grade: A-\n");
            }
            else
            {
                printf("Grade: B+\n");
            }
        }
        else
        {
            // Score is 80 or below, check if score is above 70
            if (score  >  70)
            {
                printf("Grade: C\n");
            }
            else
            {
                printf("Grade: D\n");
            }
        }
    }
    return 0;
}
```

C Switch Statement

The switch statement in C is an alternate to if-else-if ladder statement which allows us to execute multiple operations for the different possible values of a single variable called switch variable.

Here, We can define various statements in the multiple cases for the different values of a single variable.

C Switch Statement

Syntax

```
switch(expression)
{
case value1:
 //code to be executed;
 break;  //optional
case value2:
 //code to be executed;
 break;  //optional
.....

default:
 code to be executed if all cases are not matched;
}
```

C Switch Statement

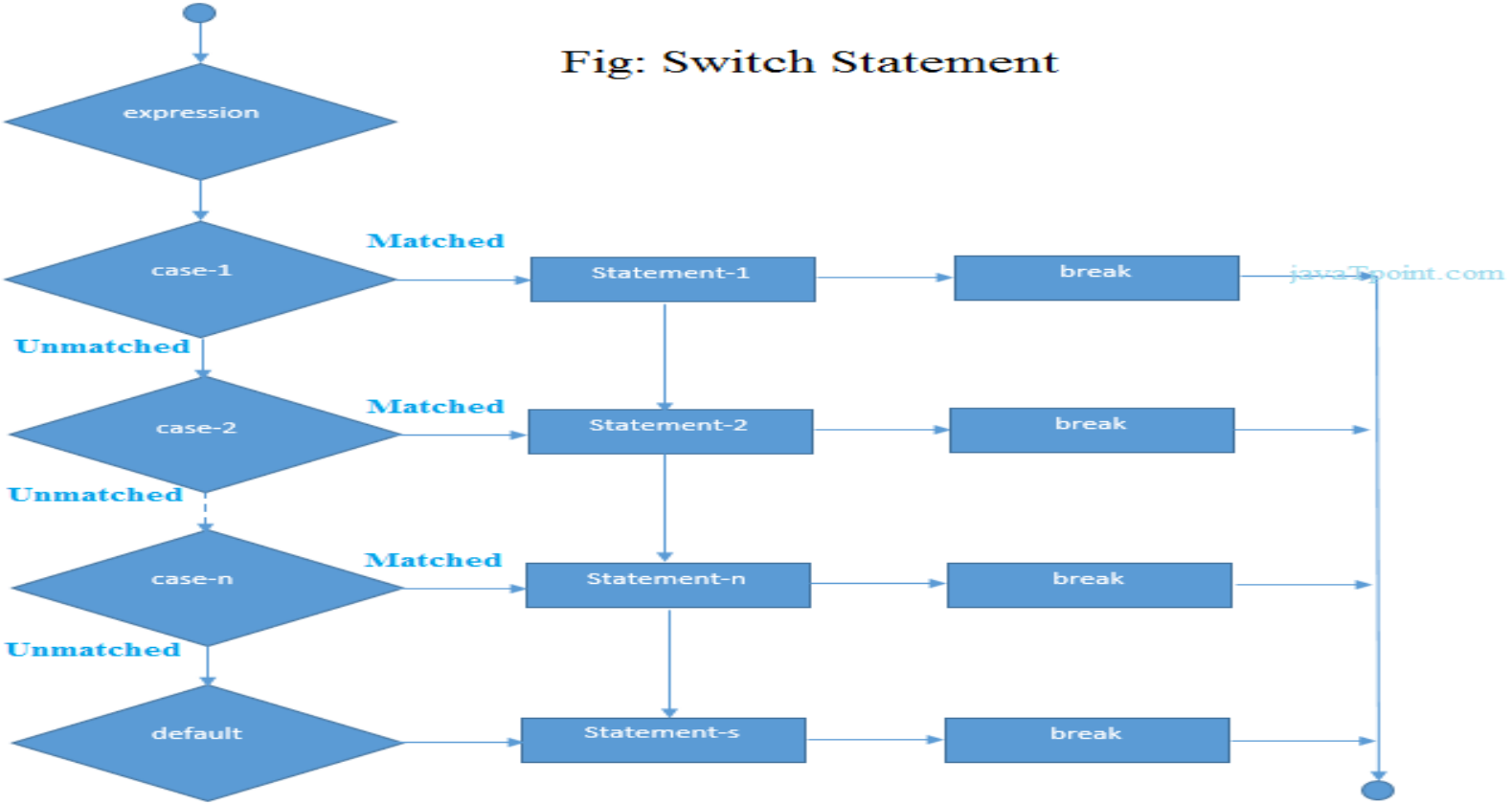
Rules for switch statement in C language

- The *switch expression* must be of an integer or character type.
- The *case value* must be an integer or character constant.
- The *case value* can be used only inside the switch statement.
- The *break statement* in switch case is not must. It is optional. If there is no break statement found in the case, all the cases will be executed present after the matched case. It is known as *fall through* the state of C switch statement.

C Switch Statement

Valid Switch	Invalid Switch	Valid Case	Invalid Case
switch(x)	switch(f)	case 3;	case 2.5;
switch(x>y)	switch(x+2.5)	case 'a';	case x;
switch(a+b-2)		case 1+2;	case x+2;
switch(func(x,y))		case 'x'>'y';	case 1,2,3;

C Switch Statement



C Switch Statement

Functioning of switch case statement

- First, the integer expression specified in the switch statement is evaluated.
- This value is then matched one by one with the constant values given in the different cases.
- If a match is found, then all the statements specified in that case are executed along with the all the cases present after that case including the default statement.
- No two cases can have similar values.
- If the matched case contains a break statement, then all the cases present after that will be skipped, and the control comes out of the switch.
- Otherwise, all the cases following the matched case will be executed.

C Switch Statement

```
#include <stdio.h>
int main()
{
  int num = 2;
  switch (num)
 {
  case 1:
        printf("Value is 1\n");
        break;
  case 2:
        printf("Value is 2\n");
        break;
case 3:
        printf("Value is 3\n");
        break;
default:

printf("Value is not 1, 2, or 3\n");
      break;
}
return 0;
}
```

Break and Default keyword in Switch statement

Break Keyword

The ***"break" keyword*** is used within the code block of each case to terminate the switch statement prematurely.

When the program encounters a ***"break" statement*** inside a case block, it immediately exits the ***switch statement***, preventing the execution of subsequent case blocks.

The ***"break" statement*** is crucial for avoiding switch statements' ***"fall-through"*** behavior.

Break and Default keyword in Switch statement

```
#include <stdio.h>
int main() {
int num = 3;

switch (num) {
case 1:
printf("Value is 1\n");
break; // Exit the switch statement after executing this case block
case 2:
printf("Value is 2\n");
break; // Exit the switch statement after executing this case block
case 3:
printf("Value is 3\n");
break; // Exit the switch statement after executing this case block
default:
printf("Value is not 1, 2, or 3\n");
break; // Exit the switch statement after executing the default case block
}

return 0;
}
```

Break and Default keyword in Switch statement

Default Keyword:

When none of the ***case constants*** match the ***evaluated expression***, it operates as a ***catch-all case***.

If no matching case exists and a ***"default" case exists***, the code block associated with the ***"default"*** case is run.

It is often used to handle circumstances where none of the stated situations apply to the provided input.

Adv. and Disadv. of the switch statement:

Advantages

- **Readability and clarity:** The ***switch statement*** provides a concise and straightforward way to express ***multiway branching*** in the code. Dealing with multiple cases can make the code more organized and easier to read than multiple ***nested if-else statements***.
- **Efficiency:** The ***switch statement*** is generally more efficient than a series of ***if-else statements*** when dealing with multiple conditions. It works as a d***irect jump table***, which makes it faster and more optimized in terms of execution time.
- **Case-based logic:** The ***switch statement*** naturally fits scenarios where the program needs to make decisions based on specific values of a single variable. It is an intuitive and straightforward way to implement case-based logic.

Adv. And Disadv. of the switch statement:

Disadvantages

- **Limited expressions:** The expression used in the ***switch statement*** must result in an ***integral value (char, int, enum)*** or a compatible data type. It cannot handle more ***complex or non-constant expressions***, limiting its ***flexibility*** in some scenarios.
- **Inability to compare ranges:** Unlike ***if-else statements***, the ***switch statement*** cannot handle ranges of values directly. Each case in the switch statement represents a specific constant value, making it challenging to handle a range of values efficiently.
- **No support for floating-point numbers:** The ***switch statement*** only accepts ***integral types (integers)*** and values from ***enums***; it does not accept floating-point numbers. It does not handle ***non-integral data types*** like ***floating-point integers***, which might be problematic in some circumstances.
- **Fall-through behavior:** ***Switch statements*** have ***"fall-through"*** behavior by default which implies that if a case does not include a ***"break" statement***, execution will ***"fall through"*** to the following case block. If not managed appropriately, this might result in unwanted behavior.
- **Duplicate code:** Using a ***switch statement*** might result in duplicate code in some circumstances, especially when numerous cases demand the same actions. If not properly managed, this might result in code duplication.
- **Nested switches can become complex:** When dealing with ***nested switch statements***, the code can become complex and less readable. It may require additional effort to understand and maintain such nested structures.

Adv. and Disadv. of the switch statement:

	if-else	switch
Definition	Depending on the condition in the 'if' statement, 'if' and 'else' blocks are executed.	The user will decide which statement is to be executed.
Expression	It contains either logical or equality expression.	It contains a single expression which can be either a character or integer variable.
Evaluation	It evaluates all types of data, such as integer, floating-point, character or Boolean.	It evaluates either an integer, or character.
Sequence of execution	First, the condition is checked. If the condition is true then 'if' block is executed otherwise 'else' block	It executes one case after another till the break keyword is not found, or the default statement is executed.
Default execution	If the condition is not true, then by default, else block will be executed.	If the value does not match with any case, then by default, default statement is executed.
Editing	Editing is not easy in the 'if-else' statement.	Cases in a switch statement are easy to maintain and modify. Therefore, we can say that the removal or editing of any case will not interrupt the execution of other cases.
Speed	If there are multiple choices implemented through 'if-else', then the speed of the execution will be slow.	If we have multiple choices then the switch statement is the best option as the speed of the execution will be much higher than 'if-else'.